

Zinto Manual Testing Guide — Part 9

Platform Overview & Behind-the-Scenes

Foundation

(Reference Only)

Zinto Platform — Manual Requirement Testing Guide

This document explains the engineering foundations that everything else in the Zinto app is built on — things like the database, the underlying data models, and the overall technical plan. These items are not separate buttons you click; they are the "plumbing" behind the screens you already tested in the other 8 guides in this series. This part is for the same reader (a business owner or product owner reviewing the project), and it should be read after finishing Parts 1–8, as a wrap-up that connects the dots.

Why this document looks different

Every other guide in this series (Parts 1–8) gave you numbered, click-through tests — "tap this button, you should see that result." This part is different on purpose. The items below are internal engineering foundations: things like "the database can handle many sellers at once" or "the system is built so new features can be added later without rebuilding everything." A non-technical person cannot click a button labelled "database architecture" — there is no such button. Instead, these foundations quietly make everything you already clicked-through in Parts 1–8 possible.

So for each item below, instead of steps, you will find a short plain-language explanation of what it means, plus a note on exactly which of the other 8 guides already proves it works — because if those guides' tests passed, these foundations are, by definition, working underneath them.

How to use this guide

You do not need to open the app for this part. Simply read each item below. Each one tells you (a) what it means in everyday terms, and (b) which guide(s) you already completed that indirectly proves it. If you passed the tests in those guides, you can consider these foundation items confirmed as well. No separate testing action is needed here.

Table of Contents

1. 2.1 Database / Schema / System Design Expectations
2. 2.2 Technical Expectations
3. 3.1 Multi-Vendor Model
4. 3.2 Multi-City Model
5. 3.3 Hyperlocal Commerce Model
6. 23.2 Backend Model - Customer Model
7. 23.2 Backend Model - Cart Model
8. 23.2 Backend Model - Product Model
9. 23.2 Backend Model - Order Model
10. 23.2 Backend Model - Category Model
11. 23.2 Backend Model - Banners Model
12. 23.2 Backend Model - Store Model
13. 23.2 Backend Model - Offer Model
14. 23.2 Backend Model - Transaction Model
15. 23.3 Additional Practical Models Implied
16. 34.1 Frontend Deliverables Discussed
17. 34.2 Backend Deliverables Discussed
18. 34.3 Action Items Discussed Across MOMs
19. 35. Final Product Owner Summary

2.1 Database / Schema / System Design Expectations

This section covers the foundational design rules the system's data storage was built around — how it copes with growth, variety, and heavy use.

The database is designed to grow with the business Ref: Req #14

What it means: The underlying data storage was planned so new kinds of information (new features, new screens, new business rules) can be added later without tearing everything down and starting over.

How this is confirmed: Every guide in this series (Parts 1–8) already relies on this — each new screen you tested (offers, scheduling, bulk settlements, add-on suggestions, etc.) is proof the storage layer flexed to support it.

The system supports many independent sellers on one platform Ref: Req #15

What it means: This is the "multi-vendor" foundation — many different shops can each run their own store on Zinto at the same time.

How this is confirmed: You already saw this in [Part 1: Customer App](#) when browsing products from different stores, and in [Part 3: Seller Panel](#) and [Part 4: Admin Panel — Catalog & Shop Management](#) when managing multiple individual sellers.

The system supports operating in more than one city Ref: Req #16

What it means: The platform can run in several different cities, each with its own sellers, delivery areas, and local rules.

How this is confirmed: You confirmed city-specific setup in

[Part 5: Admin Panel — Business Rules, Commission & Pricing](#) when reviewing city-wise settings, and in

[Part 6: Delivery/Logistics Flow](#) when checking delivery-area coverage.

The system can hold a very large number of products Ref: Req #17

What it means: The product catalog is built to handle a huge inventory (hundreds of thousands of products) without slowing down.

How this is confirmed: Confirmed by [Part 1: Customer App](#) (smooth browsing/scrolling of products) and [Part 2: Search, Discovery & Notifications](#) (fast search results across the catalog).

Products are organised into categories and subcategories Ref: Req #18

What it means: Products are grouped in a structured way (e.g. Groceries → Dairy → Milk) rather than one giant unsorted list.

How this is confirmed: Already confirmed while using the category tabs and menus in

Part 1: Customer App and while managing categories in

Part 4: Admin Panel — Catalog & Shop Management .

The system supports packaging-type categories Ref: Req #19

What it means: Beyond product categories, the system separately tracks how items are packaged (loose, boxed, bagged, etc.), which affects packaging charges.

How this is confirmed: Confirmed in Part 5: Admin Panel — Business Rules, Commission & Pricing when reviewing packaging charge settings, and in Part 8: Payments, Seller Settlements & Refunds when packaging charges appeared on an order bill.

Category structures can support special transaction-based rules if needed Ref: Req #20

What it means: The category system is flexible enough to attach special charge or commission rules to specific category types, not just a flat one-size-fits-all rule.

How this is confirmed: Confirmed indirectly in Part 5: Admin Panel — Business Rules, Commission & Pricing where category-specific commission and charge settings were tested.

The system supports the full order journey, including cancellations, refunds, returns, and complaints Ref: Req #21

What it means: The order process is built to handle not just "place an order and get it delivered," but also all the things that can go wrong or change afterward — cancelling part of an order, returning an item, requesting a refund, or replacing a damaged item.

How this is confirmed: Confirmed across Part 7: Order Lifecycle, Scheduling & Cancellations (cancellations, partial cancellations) and Part 8: Payments, Seller Settlements & Refunds (refunds and returns).

The system supports coupons, offers, subscriptions, and discounts Ref: Req #22

What it means: The data storage has a place to keep track of promotional codes, special offers, subscription plans, and discount rules.

How this is confirmed: Confirmed while applying coupons/offers in Part 1: Customer App and while setting up offers and pricing rules in Part 5: Admin Panel — Business Rules, Commission & Pricing .

The system is built to accommodate future features Ref: Req #23

What it means: New features can be added later on top of the existing foundation without needing to rebuild the core system.

How this is confirmed: This is a forward-looking design property rather than something with a visible test today; its soundness is reflected in how smoothly every feature across [Part 1](#) through [Part 8](#) works together on one shared foundation.

The system is built to handle high traffic Ref: Req #24

What it means: The platform should keep working smoothly even when many customers are using it at the same time.

How this is confirmed: Reflected in the responsiveness you experienced throughout [Part 1: Customer App](#) and [Part 2: Search, Discovery & Notifications](#) — smooth browsing and fast search results are the everyday evidence of this.

Search and discovery are built to scale as the catalog grows Ref: Req #25

What it means: The search feature is designed to stay fast and accurate even as more products and stores are added, at least on the customer-facing side.

How this is confirmed: Directly confirmed in [Part 2: Search, Discovery & Notifications](#) when testing product and store search.

Order management is built to scale with business growth Ref: Req #26

What it means: As more orders flow through the system, the tools for managing them (for sellers, admins, and delivery staff) are designed to keep working without breaking down.

How this is confirmed: Confirmed by the smooth order-management workflows you tested in

[Part 3: Seller Panel](#) , [Part 4: Admin Panel — Catalog & Shop Management](#) , and [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

2.2 Technical Expectations

This section covers how the system was engineered internally — how its different parts talk to each other and how secure and future-proof it is.

Different parts of the system work independently of each other Ref: Req #27

What it means: The customer app, seller panel, admin panel, and delivery app are built as separate pieces, so a change in one does not automatically break the others.

How this is confirmed: Evidenced by the fact that **Part 1: Customer App** , **Part 3: Seller Panel** , **Part 4: Admin Panel** , and **Part 6: Delivery/Logistics Flow** each work correctly as their own self-contained experiences.

The system's components are built to scale Ref: Req #28

What it means: Individual building blocks of the app (like the product catalog or the order system) can grow in capacity without a full redesign.

How this is confirmed: Same evidence as above — the day-to-day smoothness across **Part 1** through **Part 8** reflects a component structure that scales.

The backend is ready to connect with other systems (API-ready) Ref: Req #29

What it means: The behind-the-scenes engine that powers the app is built in a standard way so it could connect to other apps or services later if needed.

How this is confirmed: Every screen in every guide (Parts 1–8) talks to this same backend engine — their correct behaviour is the proof this connection layer works.

Logging in and account security are handled safely Ref: Req #30

What it means: Passwords, OTPs, and account sessions are protected so only the right person can access an account.

How this is confirmed: Confirmed while logging in as a customer, seller, admin, and delivery rider throughout **Part 1: Customer App** , **Part 3: Seller Panel** , **Part 4: Admin Panel** , and **Part 6: Delivery/Logistics Flow** .

Search and other lookups happen efficiently Ref: Req #31

What it means: When you search or filter for something, the system finds it quickly rather than making you wait.

How this is confirmed: Directly confirmed in **Part 2: Search, Discovery & Notifications** when testing search speed and filtering.

New features can be added without redesigning the whole database Ref: Req #32

What it means: The way data is stored was planned with room to grow, so adding a new feature does not require starting the database over from scratch.

How this is confirmed: Same as Req #23 above — a forward-looking property, indirectly evidenced by how well the many features across [Part 1](#) through [Part 8](#) already coexist.

Each type of user sees only the tools meant for their role Ref: Req #33

What it means: Customers, sellers, admins, and delivery staff each get their own separate app/panel with only the features relevant to them.

How this is confirmed: Confirmed by simply having used four separate, role-specific experiences:

[Part 1: Customer App](#) , [Part 3: Seller Panel](#) , [Part 4: Admin Panel — Catalog & Shop Management](#) / [Part 5: Admin Panel — Business Rules, Commission & Pricing](#) , and [Part 6: Delivery/Logistics Flow](#) .

3.1 Multi-Vendor Model

This section covers the core idea that Zinto is a marketplace of many shops, not just one store.

Many different sellers can set up shop on Zinto Ref: Req #34

What it means: This is the "multi-vendor" model in plain terms — Zinto works like a shopping mall where many independent shopkeepers each run their own store under one roof, instead of Zinto selling everything itself.

How this is confirmed: You already confirmed this while browsing multiple stores in [Part 1: Customer App](#) , and while onboarding/managing multiple sellers in [Part 3: Seller Panel](#) and [Part 4: Admin Panel — Catalog & Shop Management](#) .

3.2 Multi-City Model

This section covers how the platform's rules can differ from one city to another.

Business rules can be set differently per city Ref: Req #35

What it means: Commission rates, delivery charges, and how sellers/stores operate can each be configured separately for different cities, instead of one fixed rule for the whole country.

How this is confirmed: Confirmed while reviewing city-based settings in

Part 5: Admin Panel — Business Rules, Commission & Pricing , and while checking delivery coverage areas in

Part 6: Delivery/Logistics Flow .

3.3 Hyperlocal Commerce Model

This section covers the "quick commerce" idea — ordering from nearby shops for fast delivery.

Customers see nearby stores for fast, local delivery Ref: Req #36

What it means: This is the "hyperlocal" or "quick commerce" model — like Blinkit or Zepto — where the app shows you shops close to your location so delivery can be quick.

How this is confirmed: Confirmed while seeing nearby stores in Part 1: Customer App and

Part 2: Search, Discovery & Notifications , and while checking short delivery timelines in

Part 6: Delivery/Logistics Flow .

23.2 Backend Model - Customer Model

This section covers what information the system keeps on file about each customer.

Customer records store profile, address, wishlist, and order history Ref: Req #225

What it means: Behind every customer account, the system keeps their name, email, mobile number, saved addresses/locations, wishlist items, and past orders together in one place.

How this is confirmed: Confirmed while updating a profile, saving an address, adding items to a wishlist, and viewing order history in Part 1: Customer App .

23.2 Backend Model - Cart Model

This section covers what information the shopping cart keeps track of.

The cart remembers which products, quantities, prices, and store they came from Ref: Req #226

What it means: When items are added to the cart, the system keeps track of exactly which product, how many, at what price, and from which store, so checkout works correctly.

How this is confirmed: Confirmed every time items were added to and checked out from the cart in

Part 1: Customer App .

23.2 Backend Model - Product Model

This section covers what information is stored for each product listing.

Product records store details, images, pricing, stock, category, and store Ref: Req #227

What it means: Every product listing keeps its description, photos, price, how many are in stock, which category it belongs to, and which store sells it.

How this is confirmed: Confirmed while browsing product detail pages in Part 1: Customer App , and while adding/editing products in Part 3: Seller Panel and Part 4: Admin Panel — Catalog & Shop Management .

23.2 Backend Model - Order Model

This section covers what information is stored for each order.

Order records store customer, products, delivery, and payment status Ref: Req #228

What it means: Every order keeps track of who placed it, what was ordered, where it's being delivered, and whether it has been paid for.

How this is confirmed: Confirmed while placing and tracking orders in Part 1: Customer App and Part 7: Order Lifecycle, Scheduling & Cancellations , and while viewing payment status in Part 8: Payments, Seller Settlements & Refunds .

23.2 Backend Model - Category Model

This section covers what information is stored for each product category.

Category records store name, image, subcategories, and a web-friendly link Ref: Req #229

What it means: Each category has a name, a picture, a list of subcategories under it, and a short web address tag used internally for linking to it.

How this is confirmed: Confirmed while browsing categories and subcategories in [Part 1: Customer App](#) , and while managing categories in [Part 4: Admin Panel — Catalog & Shop Management](#) .

23.2 Backend Model - Banners Model

This section covers what information is stored for the promotional banners shown on the home page.

Banner records store images, links, active status, and display order Ref: Req #230

What it means: Each home-page banner keeps its image, where it should take you when tapped, whether it's currently switched on, and which order it should appear in among other banners.

How this is confirmed: Confirmed while viewing home page banners in [Part 1: Customer App](#) , and while managing home page content in [Part 4: Admin Panel — Catalog & Shop Management](#) .

23.2 Backend Model - Store Model

This section covers what information is stored for each seller's store.

Store records store shop details, ratings, and delivery radius Ref: Req #231

What it means: Each store's page keeps its shop information, customer ratings, and how far it delivers.

How this is confirmed: Confirmed while viewing a store page and its ratings in [Part 1: Customer App](#) , and while checking delivery area coverage in [Part 6: Delivery/Logistics Flow](#) .

23.2 Backend Model - Offer Model

This section covers what information is stored for discount offers and coupons.

Offer records store discount details, coupon codes, expiry, and usage limits Ref: Req #232

What it means: Each offer or coupon keeps track of how much discount it gives, its code, when it expires, and how many times it can be used.

How this is confirmed: Confirmed while applying a coupon at checkout and seeing its discount, code, and validity in [Part 1: Customer App](#) , and while creating/managing offers in

[Part 5: Admin Panel — Business Rules, Commission & Pricing](#) .

23.2 Backend Model - Transaction Model

This section covers what information is stored for each payment transaction.

Transaction records store payment method, transaction ID, and payment/refund status Ref: Req #233

What it means: Every payment made on the platform is logged with how it was paid, a unique reference number, and whether it succeeded, failed, or was refunded.

How this is confirmed: Confirmed while completing checkout payments in [Part 1: Customer App](#) , and while reviewing payment and refund records in [Part 8: Payments, Seller Settlements & Refunds](#) .

23.3 Additional Practical Models Implied

This section covers other behind-the-scenes record-keeping areas the platform's design needed beyond the core models above.

The system keeps dedicated records for sellers/vendors Ref: Req #234

What it means: Sellers have their own dedicated profile and business records, separate from customer accounts.

How this is confirmed: Confirmed by logging in and managing a shop in [Part 3: Seller Panel](#) , and by viewing seller listings in [Part 4: Admin Panel — Catalog & Shop Management](#) .

The system keeps dedicated records for subscriptions Ref: Req #235

What it means: If a seller or customer signs up for a subscription plan, the system tracks that plan separately.

How this is confirmed: Confirmed while reviewing seller subscription plans in [Part 3: Seller Panel](#) and [Part 5: Admin Panel — Business Rules, Commission & Pricing](#) .

The system keeps dedicated records for commission calculations Ref: Req #236

What it means: The percentage Zinto earns from each sale is tracked as its own concept, separate from the order and product records.

How this is confirmed: Confirmed while reviewing commission settings and how they show up on settlements in [Part 5: Admin Panel — Business Rules, Commission & Pricing](#) and [Part 8: Payments, Seller Settlements & Refunds](#) .

The system keeps dedicated records for refunds Ref: Req #237

What it means: When money is refunded to a customer, the system logs it as its own tracked event, separate from the original payment.

How this is confirmed: Confirmed while requesting and tracking a refund in [Part 8: Payments, Seller Settlements & Refunds](#) .

The system keeps dedicated records for customer complaints Ref: Req #238

What it means: When a customer raises a complaint or support ticket, it is tracked as its own record so it can be followed up on.

How this is confirmed: Confirmed while raising or viewing a support ticket/complaint in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) and [Part 8: Payments, Seller Settlements & Refunds](#) .

The system keeps dedicated records for wallet balances Ref: Req #239

What it means: Each user's in-app wallet balance (used for payments and refunds) is tracked as its own record.

How this is confirmed: Confirmed while paying with wallet balance or receiving a wallet refund in [Part 1: Customer App](#) and [Part 8: Payments, Seller Settlements & Refunds](#) .

The system keeps dedicated records for notifications Ref: Req #240

What it means: Alerts sent to customers, sellers, and admins (like order updates) are tracked and stored so they can be shown in a notifications list.

How this is confirmed: Confirmed while receiving and viewing notifications in

[Part 2: Search, Discovery & Notifications](#) .

The system keeps dedicated records for delivery/logistics Ref: Req #241

What it means: Delivery assignments, rider details, and delivery progress are tracked as their own set of records (internally called "deliveries").

How this is confirmed: Confirmed while tracking a rider and delivery progress in

[Part 6: Delivery/Logistics Flow](#) .

Seller availability tracking Ref: Req #242

How this is confirmed: This concerns whether a dedicated record tracks when a seller/store is open or closed for orders. You can observe the customer-facing effect of this (a store showing as open or closed) in

[Part 1: Customer App](#) and in store hour settings in [Part 3: Seller Panel](#) ; any gaps here would show up as a store's open/closed status not matching what was set.

Add-on product mapping Ref: Req #243

What it means: This is about how the system links a main product to suggested add-on products (e.g. suggesting butter with bread).

How this is confirmed: The customer-facing result of this is what you tested in [Part 1: Customer App](#) and

[Part 2: Search, Discovery & Notifications](#) when you saw add-on/pairing suggestions while shopping, and while managing add-ons in [Part 3: Seller Panel](#) and [Part 4: Admin Panel — Catalog & Shop Management](#) .

Advertisement / Super AD records Ref: Req #244

What it means: This is about tracking sponsored/advertised product placements separately from regular listings.

How this is confirmed: The visible effect — sponsored or promoted product placements — is what you would have seen in [Part 1: Customer App](#) and [Part 2: Search, Discovery & Notifications](#) if any promoted listings appeared during browsing or search.

Future/scheduled order configuration Ref: Req #245

What it means: This is about how the system stores the settings for products that are scheduled to be sold or delivered on a future date.

How this is confirmed: The visible result of this is what you tested in

[Part 7: Order Lifecycle, Scheduling & Cancellations](#) when placing and reviewing a scheduled order.

The system keeps dedicated records for seller settlements and payouts Ref: Req #246

What it means: The money owed to each seller, and when it was paid out, is tracked in its own dedicated ledger.

How this is confirmed: Confirmed while reviewing seller earnings, payouts, and settlement history in

[Part 8: Payments, Seller Settlements & Refunds](#) .

Bulk-order settlement tracking Ref: Req #247

What it means: This is about a dedicated record for tracking settlements on large "bulk" orders (as opposed to regular individual orders).

How this is confirmed: The visible result is the dedicated "Bulk Order Settlements" page you would have reviewed in [Part 8: Payments, Seller Settlements & Refunds](#) .

Admin-assisted shop/product management support structure Ref: Req #248

What it means: This is about whether there's a dedicated record trail for cases where the admin team directly helps set up or edit a seller's shop or products, for audit purposes.

How this is confirmed: The visible result — an admin being able to add/edit a seller's products or shop details on their behalf — is what you tested in [Part 4: Admin Panel — Catalog & Shop Management](#) .

34.1 Frontend Deliverables Discussed

This section lists the customer-facing screens that were planned and built, as agreed in project discussions.

Login and sign-up screens were delivered Ref: Req #360

How this is confirmed: Confirmed by logging in and signing up in [Part 1: Customer App](#) .

The home page was delivered Ref: Req #361

How this is confirmed: Confirmed by viewing the home page in [Part 1: Customer App](#) .

Search screens were delivered Ref: Req #362

How this is confirmed: Confirmed by using search in [Part 2: Search, Discovery & Notifications](#) .

Store discovery pages were delivered Ref: Req #363

How this is confirmed: Confirmed by browsing store listing/discovery pages in [Part 1: Customer App](#) and [Part 2: Search, Discovery & Notifications](#) .

34.2 Backend Deliverables Discussed

This section lists the behind-the-scenes engineering work that was planned and built.

The database structure was built Ref: Req #364

How this is confirmed: This is the storage layer underneath every guide in this series — its correctness is reflected in every screen across [Part 1](#) through [Part 8](#) working as expected.

The system is built with connectable, standard backend building blocks Ref: Req #365

How this is confirmed: Same as above — reflected in the correct functioning of every app/panel tested across [Part 1](#) through [Part 8](#) .

The relationships between different pieces of information were defined Ref: Req #366

What it means: For example, the system knows an order belongs to a customer, contains products, and comes from a store — these connections were properly mapped out.

How this is confirmed: Reflected in orders correctly showing the right customer, products, and store details across [Part 1: Customer App](#) and [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

34.3 Action Items Discussed Across MOMs

This section recaps action items that came out of project meetings (MOMs = Minutes of Meetings) over the course of development — most map directly onto features you already tested elsewhere.

Finalize the scheduled order workflow Ref: Req #367

How this is confirmed: Confirmed by placing/reviewing a scheduled order in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

Implement stock visibility logic Ref: Req #368

How this is confirmed: Confirmed by seeing correct in-stock/out-of-stock labelling on products in [Part 1: Customer App](#) and [Part 2: Search, Discovery & Notifications](#) .

Define the refund, cancellation, and replacement process Ref: Req #369

How this is confirmed: Confirmed by cancelling an order in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) and by requesting a refund/return in [Part 8: Payments, Seller Settlements & Refunds](#) .

Implement order tracking functionality Ref: Req #370

How this is confirmed: Confirmed by watching the order status tracker update step-by-step in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

Set up vendor onboarding via email Ref: Req #371

How this is confirmed: Confirmed by seeing the seller invite/onboarding email flow in [Part 3: Seller Panel](#) and [Part 4: Admin Panel — Catalog & Shop Management](#) .

Develop admin order management features Ref: Req #372

How this is confirmed: Confirmed by managing orders as an admin in [Part 4: Admin Panel — Catalog & Shop Management](#) and [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

Develop the home page UI Ref: Req #373

How this is confirmed: Confirmed by viewing the home page in [Part 1: Customer App](#) .

Implement search features Ref: Req #374

How this is confirmed: Confirmed by searching in [Part 2: Search, Discovery & Notifications](#) .

Design an attractive store section Ref: Req #375

How this is confirmed: Confirmed by browsing store pages in [Part 1: Customer App](#) and [Part 2: Search, Discovery & Notifications](#) .

Create database schemas for the discussed models Ref: Req #376

How this is confirmed: Reflected in every model-backed screen working correctly across [Part 1](#) through [Part 8](#) .

Define model relationships Ref: Req #377

How this is confirmed: Same as Req #366 above — reflected in correctly linked order/customer/store/product data across [Part 1](#) and [Part 7](#) .

Prepare for UI, schema, and feature-prioritization review discussions Ref: Req #378

How this is confirmed: This is a project-management/meeting-preparation action rather than a testable feature — it is reflected in the fact that a coherent, reviewable product exists across all 9 parts of this testing guide series.

35. Final Product Owner Summary

This section is a high-level wrap-up of everything the platform delivers, viewed as a whole. Each line below is a big-picture theme that ties together things you already tested in detail across Parts 1–8.

Part 1-2: Customer Experience



Part 3-4: Seller & Admin Management



Part 5: Pricing & Commission Rules



Part 6-7: Delivery & Order Lifecycle



Part 8: Payments & Settlements

Seller onboarding and flexible seller business models Ref: Req #379

How this is confirmed: Confirmed in **Part 3: Seller Panel** and **Part 4: Admin Panel — Catalog & Shop Management** .

Dynamic commission and pricing logic Ref: Req #380

How this is confirmed: Confirmed in **Part 5: Admin Panel — Business Rules, Commission & Pricing** .

Category, store, and product-level configurability Ref: Req #381

How this is confirmed: Confirmed in **Part 4: Admin Panel — Catalog & Shop Management** and **Part 5: Admin Panel — Business Rules, Commission & Pricing** .

Premium customer-facing UI Ref: Req #382

How this is confirmed: Confirmed by browsing the app throughout **Part 1: Customer App** .

Search-heavy product/store discovery Ref: Req #383

How this is confirmed: Confirmed in **Part 2: Search, Discovery & Notifications** .

Smart recommendations and add-on selling Ref: Req #384

How this is confirmed: Confirmed by seeing suggested pairings/add-ons while shopping in [Part 1: Customer App](#) , and while managing add-on suggestions in [Part 3: Seller Panel](#) and [Part 4: Admin Panel — Catalog & Shop Management](#) .

Configurable homepage and advertising support Ref: Req #385

How this is confirmed: Confirmed by viewing banners/home sections in [Part 1: Customer App](#) and managing them in [Part 4: Admin Panel — Catalog & Shop Management](#) .

Full order lifecycle visibility Ref: Req #386

How this is confirmed: Confirmed by the end-to-end order tracker in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

Flexible cancellation, refund, and replacement workflows Ref: Req #387

How this is confirmed: Confirmed in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) and [Part 8: Payments, Seller Settlements & Refunds](#) .

Real-time notifications and post-order engagement Ref: Req #388

How this is confirmed: Confirmed by receiving order/delivery notifications in [Part 2: Search, Discovery & Notifications](#) and [Part 6: Delivery/Logistics Flow](#) .

Scheduled orders and seller-driven future/pre-book product publishing Ref: Req #389

How this is confirmed: Confirmed in [Part 7: Order Lifecycle, Scheduling & Cancellations](#) .

Flexible logistics participation model for sellers Ref: Req #390

What it means: Sellers can choose how their orders get delivered (e.g. their own delivery staff vs. platform delivery riders).

How this is confirmed: Confirmed in [Part 6: Delivery/Logistics Flow](#) and delivery-related settings in [Part 3: Seller Panel](#) .

Seller settlement and payout handling Ref: Req #391

How this is confirmed: Confirmed by reviewing seller earnings and payouts in

[Part 8: Payments, Seller Settlements & Refunds](#) .

Admin-assisted product listing and seller shop setup Ref: Req #392

How this is confirmed: Confirmed by an admin adding/editing a seller's products or shop details in

[Part 4: Admin Panel — Catalog & Shop Management](#) .

Scalable architecture for multi-city, high-traffic operations Ref: Req #393

How this is confirmed: Reflected in the smooth, responsive experience across [Part 1: Customer App](#) ,

[Part 2: Search, Discovery & Notifications](#) , and the multi-city settings reviewed in

[Part 5: Admin Panel — Business Rules, Commission & Pricing](#) .

Clear role-based separation with configurable, future-ready business rules Ref: Req #394

How this is confirmed: Reflected in having four distinct, correctly working experiences for Customer, Seller, Admin, and Delivery roles across [Part 1](#) , [Part 3](#) , [Part 4](#) , [Part 5](#) , and [Part 6](#) , plus the configurable business rules reviewed in [Part 5: Admin Panel — Business Rules, Commission & Pricing](#) .